

BigTech Electronics Store

Product discovery, checkout, order coordination, and Ezzie support

Project type	Full stack Java ecommerce demonstration
Market	India
Frontend	HTML, CSS, and JavaScript
Backend	Java and Spring Boot
Prepared by	Harika

This report documents the customer journey, website design, Java order flow, failure scenarios, Ezzie support, testing, deployment, and limitations.

1. Executive summary

BigTech is a small electronics store demonstration with twenty products across phones, laptops, televisions, audio, and appliances. Prices use Indian Rupees and the interface works on desktop and mobile layouts.

The customer journey covers search, category filters, sorting, stock status, cart quantities, address validation, payment simulation, confirmation, failed payment, delivery estimate, and recent orders.

The frontend remains usable as a static hosted demonstration. A separate Spring Boot API implements request validation and a testable order coordination sequence.

2. Problem and project purpose

A believable ecommerce project needs more than attractive product cards. It must preserve cart state, prevent impossible quantities, recheck stock before payment, distinguish payment failure from order confirmation, and explain what happens next.

The project purpose is to demonstrate these connected behaviours at a final-year student scope. It is not a commercial store, a payment processor, or a live inventory system.

Success means that ordinary and failure flows both work, tests cover the business rules, and the documentation clearly separates simulated behaviour from real transactions.

3. Customer journey

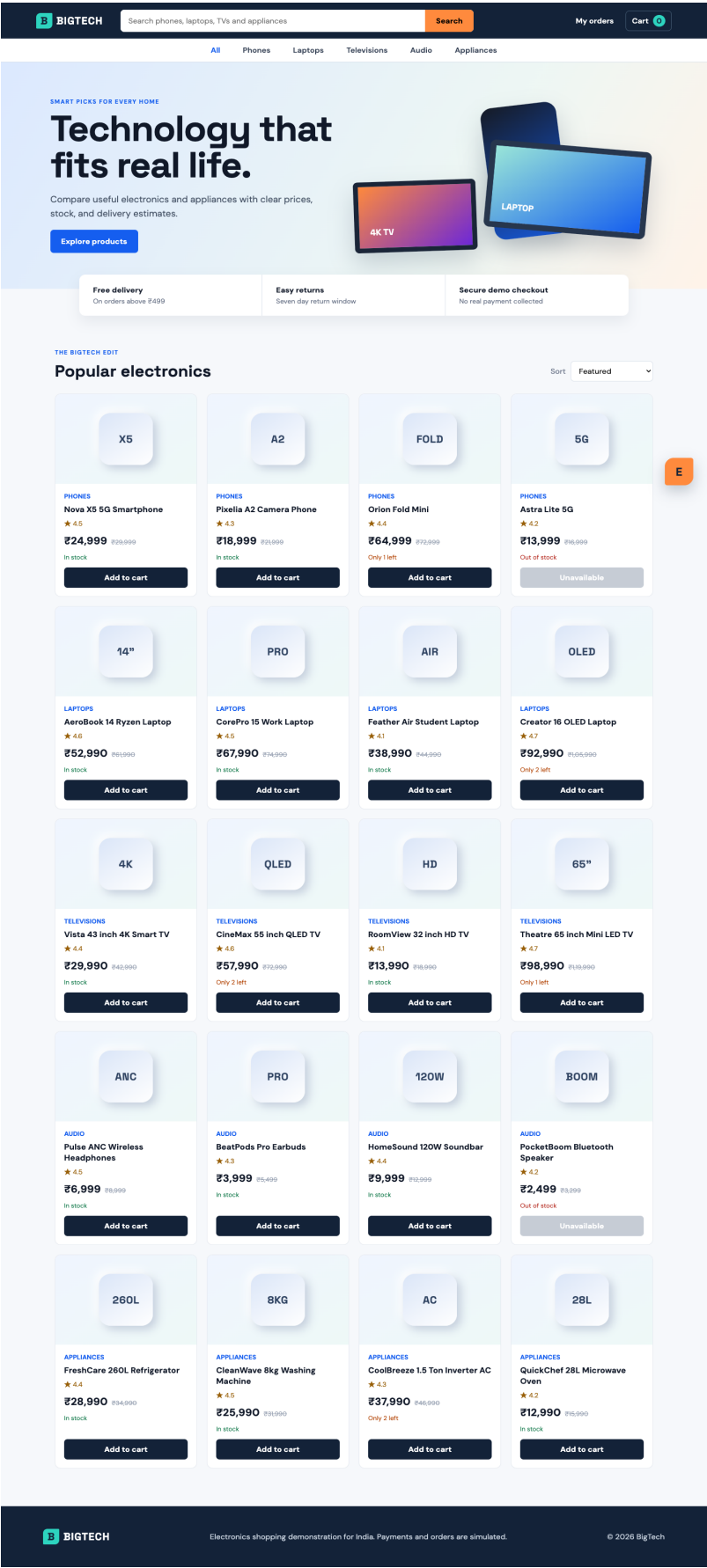
The home page introduces the store and presents categories. Search matches product names and categories, while sorting changes the catalogue without reloading the page.

Adding a product checks published demo stock. The cart supports quantity changes up to available stock and calculates subtotal, delivery, and total. Checkout validates contact and pincode fields without requesting card or UPI details.

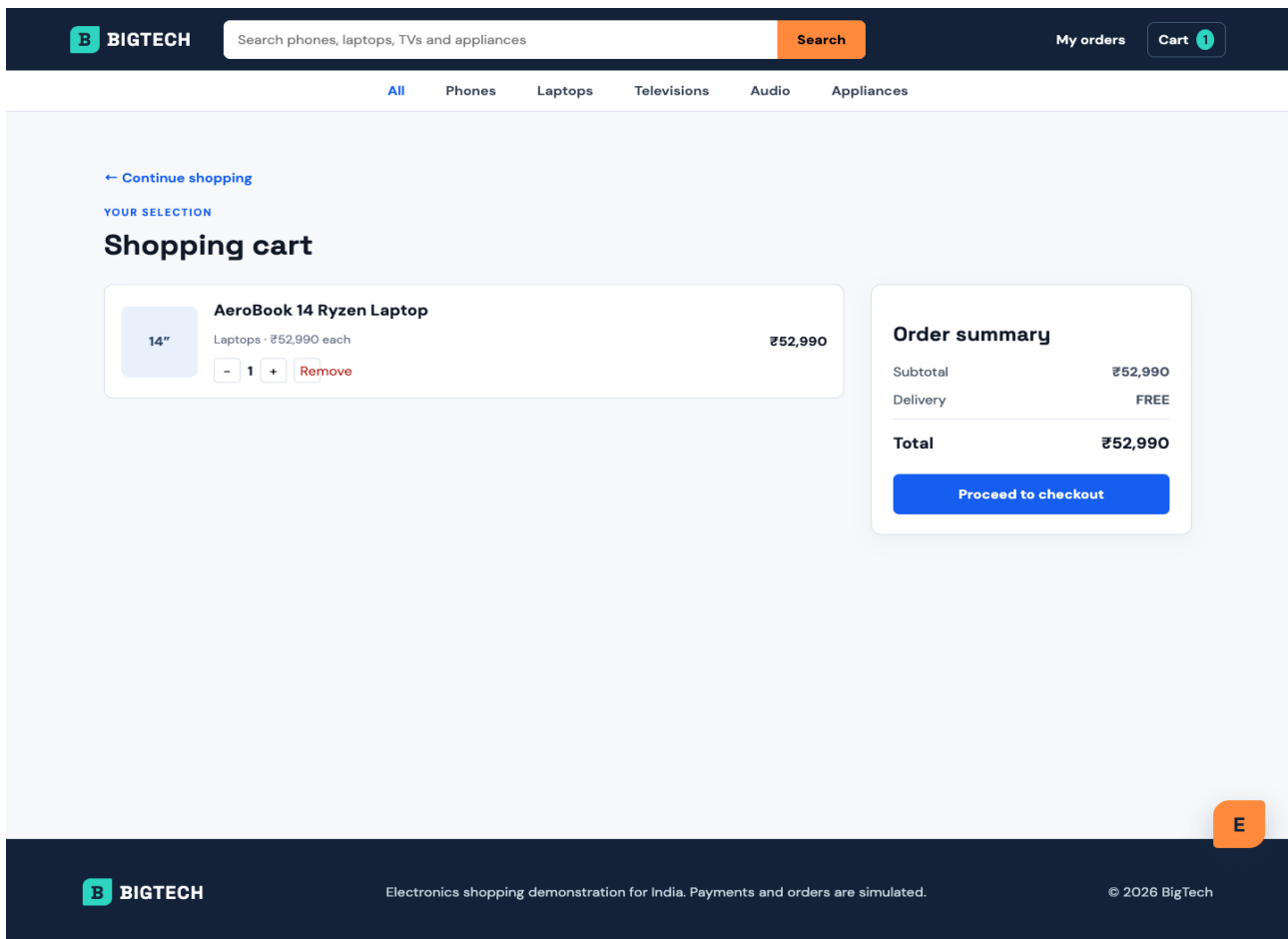
Successful payment produces an order number and delivery estimate. Failed payment creates a visible order record without a delivery date, making the negative path inspectable.

Step	Customer outcome
Browse	Search, filter, sort, and compare
Cart	Change quantity and see price totals
Checkout	Enter delivery details and simulate payment
Confirmation	Receive order number and date
Orders	Review confirmed, shipped, and failed examples

4. Storefront design



5. Cart and checkout



The cart keeps product, quantity, unit price, delivery, and total visible in one place. Checkout repeats the order summary and labels payment as a simulation. Stock is checked again when the order is created, preventing a stale cart from silently succeeding.

6. Ezzie customer support

The screenshot displays the BigTech Electronics Store website. At the top, there is a dark blue header with the BigTech logo, a search bar, and links for 'My orders' and 'Cart'. Below the header, a navigation bar lists categories: All, Phones, Laptops, Televisions, Audio, and Appliances. The main content area features a large banner with the text 'Technology that fits real life.' and a sub-headline 'SMART PICKS FOR EVERY HOME'. Below the banner, there are three boxes: 'Free delivery' (On orders above ₹499), 'Easy returns' (Seven day return window), and 'Secure delivery' (No real payment). A '4K TV' is shown in the background. Below the banner, there is a section titled 'THE BIGTECH EDIT' with the heading 'Popular electronics'. This section contains three cards: 'X5', 'A2', and 'FOLD', each with a 'PHONES' label below it. On the right side, a chatbot interface for 'Ezzie BigTech support' is open. It shows a greeting: 'Hi, I'm Ezzie. Ask me about products, stock, delivery, returns, your cart, or an order number.' Below this, there is a button 'Suggest a laptop under 60000'. The chatbot responds: 'My best match is AeroBook 14 Ryzen Laptop at ₹52,990. It is rated 4.6 and has 6 in stock.' and includes a button 'Add suggestion to cart'. At the bottom of the chatbot, there are buttons for 'Suggest a laptop', 'What is in my cart?', and 'Return policy', followed by a text input field 'Ask Ezzie about BigTech' and a 'Send' button. An orange button with the letter 'E' is visible at the bottom right of the chatbot interface.

Ezzie answers questions about BigTech products, budget, stock, cart, checkout, payment, delivery, returns, and order numbers. A product answer can add the suggested item directly to the cart. Unrelated requests receive a scope boundary instead of an invented answer.

7. Java order coordination

The Spring Boot API receives a validated order request. Inventory is checked before payment, payment outcome is evaluated, and fulfilment is scheduled only for an accepted order.

The coordinator returns explicit failure codes. STOCK_CHANGED stops a request before payment, while PAYMENT_FAILED records that inventory was available but authorisation did not succeed.

Large items such as televisions and appliances receive a four day estimate. Smaller electronics use two days. A Clock dependency makes date behaviour deterministic in tests.

Component	Responsibility
OrderController	Validate HTTP requests and return status
InventoryService	Verify requested quantity
PaymentService	Simulate authorisation
FulfilmentService	Calculate delivery date
OrderCoordinator	Apply the sequence and failure rules

8. Verification and scenarios

Five frontend unit tests cover stock, cart totals, failed payment, Ezzie scope, and budget recommendation. Three Java tests cover confirmation, stock rejection, and failed payment without a delivery date.

Browser verification checks the full storefront, opens Ezzie, adds a suggested laptop, and confirms the cart total. Evidence images are generated from the running site.

Continuous integration runs the JavaScript and Java suites on every update.

Scenario	Expected result
Out of stock item	Add button disabled
Quantity exceeds stock	Cart update blocked
Payment succeeds	Confirmed order and delivery date
Payment fails	Failed order and no delivery date
Outside Ezzie topic	Website-only boundary

9. Deployment, limitations, and next steps

The public folder is deployable as a static website. Browser local storage holds demo cart and order history, so each visitor sees state only on the current device. The Java service runs locally as an independently tested API.

The project has no authentication, database, real product feed, payment gateway, courier integration, or multi-device account. Stock resets with the demo data and order numbers are not commercial records.

Next steps are to connect the frontend to the Java API, add a small database, introduce customer authentication, validate idempotent payments, reserve stock transactionally, and deploy the backend to a Java hosting service. These additions should follow only after the current flow stays fully tested.

Deployment	Role
Static host	Public customer demonstration
Local Spring Boot	Java API demonstration
GitHub Actions	Repeat frontend and backend tests
PDF report	Explain decisions and evidence